

Farm Route Planning as a Directed Arc-Routing Problem

1 Purpose

This document specifies construction and solution of the farm-level route-planning problem.

The planner takes a farm task containing a set of fields to service, an implement configuration, a start pose, and an end pose. It produces a single ordered tractor route through crop-row service paths, field maneuvering regions, and the farm road network.

The optimization is formulated as a directed arc-routing problem and solved using OR-Tools CP-SAT. The purpose of CP-SAT is to determine which directed graph arcs the tractor traverses, and how many times each arc is traversed, such that:

1. every required farming task is serviced at least once;
2. the tractor starts at the specified start pose;
3. the tractor finishes at the specified end pose;
4. the selected traversals form one connected route;
5. all movement is through explicitly permitted graph arcs; and
6. total route distance is minimized.

The geometric generation of road arcs, crop-row arcs, and Reeds–Shepp (RS) connection arcs occurs before CP-SAT is invoked. Each graph arc therefore contains both a traversal cost and the geometric tractor path associated with that traversal.

CP-SAT determines how many times each directed graph arc is traversed. The selected arc multiplicities are then converted into an ordered tractor route by constructing the corresponding directed multigraph and computing an Euler trail from the specified start vertex to the specified end vertex. The stored geometries of the ordered arcs are concatenated to produce the farm-level tractor path.

Thus the route-generation pipeline is

1. farm geometry and task
2. directed routing graph
3. CP-SAT traversal optimization
4. directed Euler-trail reconstruction
5. tractor path

The resulting path is subsequently supplied to the vehicle trajectory-following or control subsystem for execution.

2 Farm Task

The input farm task is

$$\mathcal{F} = (\mathcal{L}, \tau, p_s, p_t),$$

where

- $\mathcal{L}$ is the list of fields to be serviced;
- τ is the field task;
- p_s is the tractor start pose;
- p_t is the tractor end pose.

For the current implementation,

$$\tau = (\text{ImplementSide}, \text{ImplementRange}),$$

with

$$\text{ImplementSide} \in \{\text{Left}, \text{Right}, \text{Both}\}.$$

The default implement range is

$$\text{ImplementRange} = \frac{\text{row spacing}}{2}.$$

A pose is an SE(2) tractor pose,

$$p = (x, y, \theta).$$

3 Logical Service Requirements

The set of farming operations that must be completed is denoted

$$\mathcal{Q} = \{q_1, q_2, \dots, q_K\}.$$

An element $q \in \mathcal{Q}$ is a *logical service requirement*. It is not itself a graph node or graph edge. For example, a requirement can mean:

Inspect this side of this tree row using the left-mounted implement.

or

Traverse a path from which this tree-row side lies within the range of an implement mounted on both sides.

The geometry generator determines which directed tractor traversals satisfy each requirement. The separation between logical tasks and physical paths is important.

Several service requirements may be satisfied by one physical tractor traversal. Conversely, the same physical path may be traversed in the opposite direction without satisfying a particular service requirement.

3.1 One-sided implements

For a left- or right-mounted implement, direction matters. Suppose a physical row path has endpoints A and B . Both physical traversals may be legal:

$$A \rightarrow B$$

and

$$B \rightarrow A.$$

However, only one of these directions may place the trees on the implement side of the tractor. Thus both arcs may exist in the travel graph, while only one is marked as satisfying the corresponding service requirement.

This permits, for example,

$$A \rightarrow B$$

to perform the farming task, followed later by

$$B \rightarrow A$$

as a deadhead traversal needed to leave the field.

3.2 Overlapping tasks

For `ImplementSide=Both`, candidate service paths associated with neighboring tree rows may overlap. Logical service requirements must not be discarded merely because their physical paths overlap. Instead, geometrically equivalent physical paths are canonicalized into a single graph arc, and that graph arc records every service requirement it satisfies.

Thus one traversal can satisfy several requirements simultaneously.

4 Directed Routing Graph

Construct a directed multigraph

$$G = (V, A).$$

Multiple directed arcs between the same pair of graph vertices are permitted.

Each vertex represents a routing state. Each directed arc represents one physically permitted tractor movement. Partition the arc set as

$$A = A_{\text{road}} \cup A_{\text{row}} \cup A_{\text{RS}}.$$

Additional arc types may be introduced later without changing the optimization formulation. Each arc $a \in A$ stores at least

$$a = (u_a, v_a, \gamma_a, \ell_a, T_a, F_a, C_a),$$

where

- $u_a \in V$ is the tail vertex;
- $v_a \in V$ is the head vertex;
- γ_a is the geometric path followed by the tractor;
- ℓ_a is the path length in meters;
- T_a is the arc type;
- F_a is an optional field identifier; and
- $C_a \subseteq \mathcal{Q}$ is the set of service requirements automatically satisfied when this arc is traversed.

For ordinary road and RS transit arcs,

$$C_a = \emptyset.$$

For crop-row traversals, C_a may contain one or more service requirements.

5 Graph Vertices and Tractor Pose

A graph vertex is a routing state rather than necessarily a unique (x, y) location.

Where heading matters, especially at row endpoints, two vertices may occupy the same position while having different headings.

For example, the same physical row endpoint can have states

$$(x, y, \theta)$$

and

$$(x, y, \theta + \pi).$$

These must not be merged because they represent different tractor states and have different RS connections.

The initial implementation may treat ordinary road intersections as position-only routing states and leave detailed road-intersection turn feasibility to the downstream trajectory planner. Field row endpoints and field/road RS interfaces must preserve the heading required by the RS calculation.

6 Road Graph Construction

The road network exists independently of the set of fields being serviced.

Split road geometry at

- road intersections;
- field-road connection points;
- StartPose or EndPose projections onto a road; and
- other topology changes.

A bidirectional road section between vertices u and v produces two directed arcs:

$$u \rightarrow v$$

and

$$v \rightarrow u.$$

Both arcs normally have cost equal to the road polyline length.

A one-way road creates only the permitted directed arc. Road arcs remain available even when the road passes through a field that is not in the current `field_list`.

No non-road field transit is created in an unselected field.

7 Crop-Row Traversal Arcs

For every selected field, generate tractor-path centerlines from the tree-row geometry and the implement configuration. Candidate tractor paths are obtained by offsetting the appropriate tree-row centerline by `ImplementRange`. Each canonical physical row path produces directed traversal arcs in each physically legal direction.

For a physical path with endpoint states u and v , these may be

$$a^+ : u \rightarrow v$$

and

$$a^- : v' \rightarrow u',$$

where u' and v' represent the opposite-heading tractor states at the same physical endpoint locations.

The service-coverage sets

$$C_{a^+} \quad \text{and} \quad C_{a^-}$$

are determined by implement geometry.

For a one-sided implement, one direction may satisfy a service requirement while the reverse direction is merely a legal transit traversal.

For a two-sided implement, one physical traversal may satisfy several service requirements simultaneously.

7.1 Canonicalization of overlapping row paths

Candidate row paths that describe the same physical directed tractor trajectory should be merged into one physical arc.

Path equivalence should use configurable tolerances for

- endpoint position;
- endpoint heading; and
- path geometry.

When equivalent candidate arcs are merged, their coverage sets are unioned:

$$C_a = C_{a_1} \cup C_{a_2} \cup \dots$$

Do not merge arbitrary graph vertices merely because their Euclidean positions are close. Vertex merging means the tractor may transition between those routing states with zero movement and is therefore a semantic operation, not merely geometric deduplication.

8 Reeds–Shepp Connection Arcs

For each selected field, form a set of field interface states consisting of appropriate

- row endpoint poses;
- field-road connection poses;
- StartPose, if located in the field; and
- EndPose, if located in the field.

For candidate ordered pairs of interface states (p_i, p_j) in the same selected field:

1. compute the Reeds–Shepp path from p_i to p_j ;
2. compute its geometric length;
3. check the complete path against the field’s permitted driving region;
4. preferably check the swept tractor footprint rather than only the tractor reference point;
5. reject the connection if it leaves the permitted region; and
6. otherwise add a directed RS arc to G .

Thus

$$a_{ij}^{\text{RS}} \in A$$

if and only if the corresponding RS maneuver is considered physically legal. An RS arc stores the actual RS geometry γ_a , not merely its scalar distance.

The first implementation should treat the complete selected field as the maneuvering region, including headlands and sidelands, subject to explicitly forbidden regions.

For a field not contained in the farm task’s field list,

$$A_{\text{row}} = \emptyset, \quad A_{\text{RS}} = \emptyset$$

for that field. Only its road network remains usable.

8.1 RS candidate pruning

The mathematically complete graph considers every relevant ordered endpoint pair in a selected field. A practical implementation may use a configured maximum Euclidean separation

$$d_{\text{candidate}}(i, j) \leq R_{\text{RS}}$$

to avoid generating obviously undesirable long-range RS connections. Such pruning changes the feasible graph and therefore must be treated as an explicit planning policy rather than as an invisible optimization approximation.

9 Start and End

The graph must contain distinguished vertices

$$s \in V$$

and

$$t \in V$$

representing StartPose and EndPose.

If a pose lies on a road, split the road geometry and insert the pose into the road graph. If a pose lies inside a selected field, include it in the appropriate RS interface-state set. If a pose cannot legally connect to the constructed graph, graph construction fails before CP-SAT is invoked.

The special case

$$s = t$$

represents a closed mission.

10 Arc Costs

Initially, the optimization objective is total geometric distance.

For each arc a ,

$$\ell_a = \text{length}(\gamma_a).$$

CP-SAT uses integer coefficients. Therefore convert distances into integer units before model construction.

For example, using centimeters,

$$c_a = \text{round}(100 \ell_a).$$

The optimization cost therefore differs from true metric distance by at most the chosen quantization resolution.

The model should retain ℓ_a in floating-point meters for reporting and route reconstruction while using c_a for CP-SAT.

All real movement arcs should have positive cost.

Future versions may extend

$$c_a$$

with penalties for reversing, undesirable surfaces, gear changes, field exits, or other operational considerations. These extensions do not change the graph formulation.

11 CP-SAT Decision Variables

For every directed arc $a \in A$, define an integer traversal-count variable

$$x_a \in \{0, 1, \dots, M_a\}.$$

The interpretation is

x_a = number of times the tractor traverses arc a .

Road arcs, RS arcs, and row arcs may all be traversed more than once. A repeated row traversal is therefore represented naturally by

$$x_a > 1$$

or by use of both directional row arcs.

The model must provide a finite upper bound M_a because CP-SAT integer variables are bounded. For the initial implementation a conservative common bound may be used, for example

$$M_a = 2K + 2,$$

where

$$K = |\mathcal{Q}|.$$

This bound should be configurable, and the implementation should report if an optimal or feasible solution saturates an arc bound, since saturation may indicate that the bound is too restrictive.

12 Service Constraints

For every service requirement $q \in \mathcal{Q}$, define

$$A(q) = \{a \in A : q \in C_a\}.$$

The required-service constraint is

$$\sum_{a \in A(q)} x_a \geq 1 \quad \forall q \in \mathcal{Q}.$$

Thus every farming task must be serviced at least once. This formulation deliberately permits repeated service-path traversal. It also permits one physical traversal to satisfy several requirements when the same arc belongs to several $A(q)$ sets.

If

$$A(q) = \emptyset$$

for any required task, graph construction has produced an infeasible mission and CP-SAT should not be invoked.

13 Vehicle Flow Constraints

For a vertex v , define

$$\delta^+(v) = \{a \in A : u_a = v\},$$

and

$$\delta^-(v) = \{a \in A : v_a = v\}.$$

For an open route with $s \neq t$, define

$$b_v = \begin{cases} +1, & v = s, \\ -1, & v = t, \\ 0, & \text{otherwise.} \end{cases}$$

Impose

$$\sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a = b_v \quad \forall v \in V.$$

For a closed route, $s = t$, use

$$b_v = 0 \quad \forall v.$$

These constraints provide the degree conditions required for the selected directed multigraph to possess an Euler trail from s to t , provided the selected graph is also connected.

14 Connectivity Constraints

Flow balance alone is insufficient because CP-SAT could otherwise construct multiple disconnected route components.

Introduce a binary variable

$$z_v \in \{0, 1\}$$

for every vertex, where

$$z_v = 1$$

means the vertex participates in the selected route.

For every incident arc a at vertex v , impose

$$x_a \leq M_a z_v.$$

Also impose

$$z_v \leq \sum_{a \in \delta^+(v) \cup \delta^-(v)} x_a.$$

Set

$$z_s = 1.$$

For a nontrivial open mission also set

$$z_t = 1.$$

Next introduce an artificial integer connectivity-flow variable

$$f_a \in \{0, \dots, |V| - 1\}$$

for every arc. This flow has no physical interpretation. Its only purpose is to prove that every selected route vertex is reachable from the start vertex.

Artificial flow may use an arc only if the tractor uses that arc:

$$f_a \leq (|V| - 1)x_a \quad \forall a \in A.$$

Every selected vertex other than the root consumes one unit of artificial flow:

$$\sum_{a \in \delta^-(v)} f_a - \sum_{a \in \delta^+(v)} f_a = z_v \quad \forall v \neq s.$$

The start vertex supplies the total required artificial flow:

$$\sum_{a \in \delta^+(s)} f_a - \sum_{a \in \delta^-(s)} f_a = \sum_{v \in V \setminus \{s\}} z_v.$$

These constraints force all used vertices into one route component rooted at StartPose.

15 Objective

The initial objective is

$$\min \sum_{a \in A} c_a x_a$$

where c_a is the integer-scaled geometric path length.

Unlike a TSP-style formulation, crop-row lengths are included in the objective.

This is necessary because a row may be traversed more than once, and different physical service paths may have different lengths.

16 Complete CP-SAT Model

The optimization problem is therefore

$$\begin{aligned}
& \min \quad \sum_{a \in A} c_a x_a \\
& \text{subject to} \quad \sum_{a \in A(q)} x_a \geq 1 & \forall q \in \mathcal{Q}, \\
& \quad \sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a = b_v & \forall v \in V, \\
& \quad x_a \leq M_a z_{u_a} & \forall a \in A, \\
& \quad x_a \leq M_a z_{v_a} & \forall a \in A, \\
& \quad z_v \leq \sum_{a \in \delta^+(v) \cup \delta^-(v)} x_a & \forall v \in V, \\
& \quad f_a \leq (|V| - 1) x_a & \forall a \in A, \\
& \quad \sum_{a \in \delta^-(v)} f_a - \sum_{a \in \delta^+(v)} f_a = z_v & \forall v \neq s, \\
& \quad \sum_{a \in \delta^+(s)} f_a - \sum_{a \in \delta^-(s)} f_a = \sum_{v \neq s} z_v, \\
& \quad x_a \in \{0, \dots, M_a\}, & \forall a \in A, \\
& \quad z_v \in \{0, 1\}, & \forall v \in V, \\
& \quad f_a \in \{0, \dots, |V| - 1\}, & \forall a \in A.
\end{aligned}$$

This is the complete first-version mathematical optimization model.

17 CP-SAT Construction Procedure

The implementation shall perform the following steps in order.

1. Parse and validate the FarmTask.
2. Generate the logical service requirement set $\mathcal{Q}$.
3. Generate candidate row-path geometry from tree-row geometry and ImplementRange.
4. Canonicalize geometrically duplicate physical row paths while retaining all logical service requirements.
5. Construct directed row traversal arcs and their service coverage sets C_a .
6. Construct the directed road graph.
7. Insert road/field interface points.
8. Insert StartPose and EndPose into the appropriate graph geometry.
9. For each selected field, generate feasible RS arcs between candidate oriented interface states.
10. Reject every RS candidate that violates the field driving-region constraint.
11. Verify that every service requirement has at least one compatible service arc.
12. Assign integer distance costs c_a .

13. Create CP-SAT traversal variables x_a .
14. Add service constraints.
15. Add start/end vehicle-flow constraints.
16. Add route-connectivity variables and constraints.
17. Add the total-distance objective.
18. Invoke CP-SAT.
19. If the solver status is neither FEASIBLE nor OPTIMAL, report route-generation failure.
20. Extract all nonzero traversal counts x_a .
21. Reconstruct an ordered Euler trail.
22. Convert the ordered arc sequence into the final geometric route.
23. Validate the resulting route independently of CP-SAT.

18 Approximate Python Model Construction

The implementation should correspond closely to the following structure.

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()

# Traversal counts

x = {
    a.id: model.new_int_var(0, a.max_uses, f"x_{a.id}")
    for a in arcs
}

# Service requirements

for q in service_requirements:
    compatible = [a for a in arcs if q.id in a.service_ids]
    model.add(sum(x[a.id] for a in compatible) >= 1)

# Vehicle flow

for v in vertices:
    outflow = sum(x[a.id] for a in outgoing[v.id])
    inflow = sum(x[a.id] for a in incoming[v.id])

    if start_id != end_id and v.id == start_id:
        rhs = 1
    elif start_id != end_id and v.id == end_id:
        rhs = -1
    else:
        rhs = 0

    model.add(outflow - inflow == rhs)
```

```

# Used-vertex variables

z = {
v.id: model.new_bool_var(f"z_{v.id}")
for v in vertices
}

for a in arcs:
model.add(x[a.id] <= a.max_uses * z[a.u])
model.add(x[a.id] <= a.max_uses * z[a.v])

for v in vertices:
incident = incoming[v.id] + outgoing[v.id]
model.add(z[v.id] <= sum(x[a.id] for a in incident))

model.add(z[start_id] == 1)
if end_id != start_id:
model.add(z[end_id] == 1)

# Artificial connectivity flow

FMAX = len(vertices) - 1

f = {
a.id: model.new_int_var(0, FMAX, f"f_{a.id}")
for a in arcs
}

for a in arcs:
model.add(f[a.id] <= FMAX * x[a.id])

for v in vertices:
fin = sum(f[a.id] for a in incoming[v.id])
fout = sum(f[a.id] for a in outgoing[v.id])

if v.id != start_id:
model.add(fin - fout == z[v.id])

root_demand = sum(
z[v.id] for v in vertices if v.id != start_id
)

root_in = sum(f[a.id] for a in incoming[start_id])
root_out = sum(f[a.id] for a in outgoing[start_id])

model.add(root_out - root_in == root_demand)

# Integer distance objective

model.minimize(
sum(a.cost_mm * x[a.id] for a in arcs)
)

```

```
solver = cp_model.CpSolver()
status = solver.solve(model)
```

Production code should separate graph construction from CP-SAT construction. CP-SAT-specific code must not contain GIS or Reeds–Shepp geometry calculations.

19 Solver Output

CP-SAT returns traversal multiplicities

$$x_a^*$$

rather than an ordered route.

Construct a new directed multigraph

$$G^*$$

by inserting exactly x_a^* copies of every arc a for which

$$x_a^* > 0.$$

For example, if

$$x_a^* = 3,$$

the optimized route contains three distinguishable copies of that physical arc.

The flow and connectivity constraints guarantee the graph structure needed for an Euler trail from s to t .

20 Route Reconstruction

Use Hierholzer’s algorithm, or an equivalent directed Euler-trail implementation, on G^* beginning at s . The result is an ordered arc sequence

$$R = (a_1, a_2, \dots, a_N)$$

such that

$$v_{a_i} = u_{a_{i+1}}$$

for every adjacent pair and every selected arc copy occurs exactly once.

For an open mission,

$$u_{a_1} = s, \quad v_{a_N} = t.$$

For a closed mission, the Euler trail is an Euler circuit rooted at s .

The geometric tractor route is

$$\Gamma = \gamma_{a_1} \oplus \gamma_{a_2} \oplus \dots \oplus \gamma_{a_N},$$

where $\oplus$ concatenates consecutive path geometries while removing duplicate endpoint samples.

21 Service Annotation of the Reconstructed Route

While traversing the ordered arc sequence, maintain a set

$$Q_{\text{done}}.$$

Whenever an arc a_i is encountered, add

$$C_{a_i}$$

to this set.

Thus,

$$Q_{\text{done}} \leftarrow Q_{\text{done}} \cup C_{a_i}.$$

At the end of reconstruction, verify

$$Q_{\text{done}} = \mathcal{Q}.$$

If an arc occurs more than once, the first compatible occurrence may be annotated as the servicing pass and later occurrences may be marked as deadhead traversals.

For the current inspection-task model, any compatible traversal is assumed to perform service automatically.

A future implementation may introduce explicit implement-on/off decision variables if traversal and service need to be distinguished operationally.

22 Route Output

The primary route output should retain arc semantics rather than immediately reducing the result to a single point array.

For example,

```
Route:
start_pose
end_pose
total_distance_m
optimality_status
segments:
- arc_id
  arc_type
  field_id
  geometry
  distance_m
  service_ids
  occurrence_index
```

A derived continuous path may then be generated by concatenating segment geometry. This path can subsequently be resampled and converted to a ROS `nav_msgs/Path` or another representation required by the trajectory-following layer. The farm route optimizer determines the desired topological/geometric route. It does not replace the downstream vehicle trajectory controller.

23 Required Validation

After solving and route reconstruction, independently check:

1. The first route state agrees with `StartPose`.
2. The final route state agrees with `EndPose`.
3. Every consecutive pair of route segments is graph-connected.
4. Every service requirement is covered.
5. Every selected RS path lies within its allowed driving region.
6. No row or field-transit arc appears in an unselected field.
7. Arc occurrence counts in the reconstructed route equal the CP-SAT values x_a^* .
8. The reconstructed route distance equals

$$\sum_a \ell_a x_a^*$$

within numerical tolerance.

9. The integer CP-SAT objective agrees with

$$\sum_a c_a x_a^*.$$

10. No route arc violates its directionality or service semantics.

24 Minimum Unit Tests

The implementation must include at least the following synthetic tests.

24.1 Single required row

One service row is connected to the start/end road network.

Verify that the row is serviced and that repeated traversal is permitted when required to return to the endpoint.

24.2 One-sided implement

The row is physically traversable in both directions, but only one direction satisfies the service requirement.

Verify that CP-SAT chooses the service-compatible direction at least once.

Verify that the opposite direction may still appear as deadhead travel.

24.3 Shared two-sided service path

Two logical service requirements are covered by one physical directed row arc.

Verify that one traversal satisfies both requirements.

24.4 Road bridge used repeatedly

Several required tasks are accessible only through one road segment.

Verify that CP-SAT may assign

$$x_a > 1$$

to that road arc.

24.5 Unselected-field shortcut

Create a geometrically shorter route through the interior of a field not present in `field_list`.

Because no field-interior arcs are created there, verify that the optimizer uses the road network instead.

24.6 Invalid RS connection

Create an RS candidate whose path leaves the allowed field polygon.

Verify that the arc is absent from the optimization graph.

24.7 Open mission

Use distinct `StartPose` and `EndPose`.

Verify that the reconstructed Euler trail begins at s and terminates at t .

24.8 Closed mission

Set `StartPose` equal to `EndPose`.

Verify that the reconstructed route is an Euler circuit.

25 Design Boundary

The route-planning subsystem consists of four explicit stages:

1. GIS/task geometry
2. directed routing graph
3. CP-SAT arc optimization
4. Euler route reconstruction

The geometric graph builder decides what movements are physically and semantically legal. CP-SAT decides how many times each legal movement is used. Euler reconstruction converts those traversal counts into a specific ordered route. A downstream trajectory-following subsystem is responsible for executing that route on the tractor.

These responsibilities must remain separate in the implementation.